

# Connecting IdentityIQ to Google Analytics: Step-by-Step Guide

---

This guide walks you through connecting SailPoint IdentityIQ (IIQ) to Google Analytics (GA) from scratch, so that IIQ can **see** who has GA access and **grant or remove** that access when someone requests it.

You don't need to know how the pieces work inside. Every step says what to do and what you should see. If you want the details, read the [Technical Reference](#).

**Time needed:** about 1 to 2 hours the first time.

---

## Before you start

---

You need:

- ☐ IdentityIQ 8.4 running, and you can log in as an admin (e.g. `spadmin` )
- ☐ A Google account that is an **Administrator** on the Google Analytics account you want to manage
- ☐ Access to **Google Cloud Console** ([console.cloud.google.com](https://console.cloud.google.com))
- ☐ OpenSSL (it comes with Git for Windows: open *Git Bash*)
- ☐ Your **GA account ID**: in GA, open *Admin* → *Account settings*. It's a number like `410795087` .
- ☐ A **spare real Google account** (e.g. a second Gmail) to test with

Everywhere this guide shows `410795087` , use **your** GA account ID.

---

## Part 1: Set up Google

---

### Step 1. Create a Google Cloud project and turn on the API

1. Go to [console.cloud.google.com](https://console.cloud.google.com) and create a new project (any name).
2. Open **APIs & Services** → **Library**, search for **Google Analytics Admin API**, and click **Enable**.

✓ The API page shows "API enabled".

### Step 2. Create a service account

A service account is a robot Google account that IIQ will log in as.

1. Go to **IAM & Admin** → **Service Accounts** → **Create service account**.
2. Give it a name, e.g. `iiq-ga` . Skip the optional permission steps and click **Done**.
3. Copy its email. It looks like `iiq-ga@your-project.iam.gserviceaccount.com` .

### Step 3. Download a key and protect it with a password

1. Click the service account, open **Keys** → **Add key** → **Create new key** → **JSON**, and save the file somewhere safe, **outside** any Git repo (e.g. `C:\keys\ga.json`).
2. IIQ wants the key as a password-protected PEM file. In **Git Bash**:

```
bash cd /c/keys # pull the private key out of the JSON file python -c "import
json;print(json.load(open('ga.json'))['private_key'])" > ga-key.pem # encrypt it with a password (you'll
be asked to choose one; remember it) openssl pkcs8 -topk8 -v2 aes-256-cbc -in ga-key.pem -out ga-key-
encrypted.pem # delete the unprotected copy rm ga-key.pem
```

✓ `ga-key-encrypted.pem` starts with `-----BEGIN ENCRYPTED PRIVATE KEY-----`.

⚠ Never commit the key or its password to Git.

### Step 4. Give the service account Administrator in Google Analytics

1. In **Google Analytics**, go to **Admin** → **Account access management** → **+** → **Add users**.
2. Paste the service account email, choose **Administrator**, and click **Add**.

Administrator is needed because IIQ will add and remove *other* people's access.

## Part 2: Import the IIQ objects

The repo includes ready-made IIQ configuration files.

### Step 5. Edit the files for your environment

Open `config/Application/GoogleAnalytics.xml` in a text editor and change:

| Find                                                               | Change to                  |
|--------------------------------------------------------------------|----------------------------|
| <code>410795087</code> (appears several times)                     | your GA account ID         |
| <code>iiq-ga-reader@api-test-345702.iam.gserviceaccount.com</code> | your service account email |

### Step 6. Import, in this order

In IIQ: **gear icon (Global Settings)** → **Import from File**. Import each file, one at a time, **in this order**:

1. `config/Rule/Rule-GoogleAnalytics-BuildAccessBinding.xml`
2. `config/CorrelationConfig/GoogleAnalyticsCorrelation.xml`
3. `config/Application/GoogleAnalytics.xml`
4. `config/TaskDefinition/Agg-GoogleAnalytics.xml`

The order matters: the application refers to the rule and the correlation, so those must exist first.

✓ Each import says it succeeded, with no errors.

## Step 7. Add the private key

1. Go to **Applications** → **Application Definition** → **Google Analytics** → **Configuration**.
2. **Private Key**: open `ga-key-encrypted.pem` in Notepad, copy **everything** (including the BEGIN/END lines), and paste.
3. **Private Key Password**: the password you chose in Step 3.
4. Click **Test Connection**.

✓ "Test Connection Successful".

✗ **503 Service Unavailable**: almost always a wrong key password. Re-enter it.

5. Click **Save** at the **bottom of the application page**.

💡 **Important habit**: IIQ only stores application changes when you click **Save** on the main application page. Saving inside a sub-screen (like an operation) is not enough.

## Step 8. Check that everything is there

Reopen the application and check:

- ☐ **Configuration** → **Connector Operations** lists 6 operations, all **Configured**: Test Connection, Account Aggregation, Create Account, Add Entitlement, Remove Entitlement, Delete Account
- ☐ **Provisioning Policies** has a Create Account policy with a `user` field
- ☐ **Correlation** shows `user` → `email`

If something is missing, see [Appendix A](#) to add it by hand.

---

## Part 3: Load the data

### Step 9. Run the aggregation

1. Go to **Setup** → **Tasks**, find **Agg-GoogleAnalytics**, and click **Run Now**.
2. Wait for it to finish, then open the task result.

✓ The result shows accounts scanned/created greater than 0 and no errors.

3. Check **Applications** → **Google Analytics** → **Accounts**. You should see one account per GA user, with their email and roles (e.g. `predefinedRoles/viewer`).

### Step 10. Make the roles requestable

1. Go to **Applications** → **Entitlement Catalog** and filter by application **Google Analytics**.
2. For each role (e.g. `predefinedRoles/viewer`), open it, tick **Requestable**, optionally add a friendly display name like "GA Viewer", and save. - Is a role missing? Only roles someone already has show up. Click **Add** to create it: application *Google Analytics*, attribute `roles`, value such as `predefinedRoles/analyst`.
3. Go to **Setup** → **Tasks** and run **Full Text Index Refresh**.

- ✓ The roles are listed and marked Requestable.

Without the Full Text Index Refresh, roles won't show up in Manage User Access, even when they're requestable.

## Part 4: Test that it works

### Step 11. Prepare a test person

1. Pick a test identity in IIQ that does **not** have Google Analytics access yet.
2. Set their **Email** to your **spare real Google account**.

⚠ GA can only add **real Google accounts**. A made-up email gives **404 Not Found**.

### Step 12. Grant access (tests Create Account)

1. Go to **Manage User Access**, choose the test person, open **Add Access**, and find `predefinedRoles/viewer`.
2. Submit, then approve the request (*My Work* → *Approvals*).

- ✓ In GA, the test email appears with **Viewer**.
- ✓ In IIQ, the test person has a Google Analytics account named `accounts/.../accessBindings/...`.

### Step 13. Add a second role (tests Add Entitlement)

1. Request `predefinedRoles/analyst` for the same person and approve it.

- ✓ In GA, they have **both** Viewer and Analyst. If only Analyst shows, run **Agg-GoogleAnalytics** and try again.

### Step 14. Remove a role (tests Remove Entitlement)

1. Run **Agg-GoogleAnalytics**, then **Refresh Identity Cube** with the option **Refresh identity entitlements for all links** ticked. This keeps IIQ's screens up to date.
2. Go to **Manage User Access** → **Remove Access**, remove `predefinedRoles/viewer`, and approve.

- ✓ In GA, only **Analyst** is left.

### Step 15. Remove the last role (removes the user)

1. Remove `predefinedRoles/analyst` the same way.

- ✓ The test email is **gone** from GA.

Then run **Agg-GoogleAnalytics** so IIQ removes the account on its side too.

🎉 **You're done.** IIQ can now grant and remove Google Analytics access.

## Everyday routine

---

After any GA change made outside IIQ, or on a schedule (e.g. nightly), run these in order:

1. **Agg-GoogleAnalytics**: pulls the latest from GA
  2. **Refresh Identity Cube** (with *Refresh identity entitlements for all links*): updates identities
  3. **Full Text Index Refresh**: updates what's searchable in Manage User Access
- 

## Common problems

---

| What you see                                                         | What it means                                                     | What to do                                                                                                                         |
|----------------------------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Test Connection: 503                                                 | Key password wrong                                                | Re-enter the private key password                                                                                                  |
| 404 Not Found when granting access                                   | The email isn't a Google account                                  | Use a real Google email on the identity                                                                                            |
| 403 when granting access                                             | Service account isn't GA Administrator, or the scope is read-only | Step 4, and check that the scope doesn't end in <code>.readonly</code> ( <a href="#">Appendix B</a> )                              |
| "Native identity is neither present in the plan nor in the response" | The Create Account mapping is misspelled                          | The mapping must say <code>bindingName</code> (lowercase b). Don't resubmit; run aggregation instead.                              |
| Request says <b>Verifying</b>                                        | Normal: IIQ is waiting to confirm                                 | Run Agg-GoogleAnalytics, then <b>Perform Identity Request Maintenance</b>                                                          |
| Role not found in Manage User Access                                 | Search index is out of date                                       | Run <b>Full Text Index Refresh</b>                                                                                                 |
| Remove Access shows a role the person no longer has                  | Identity not refreshed                                            | Run Refresh Identity Cube with <i>Refresh identity entitlements for all links</i>                                                  |
| A new GA account landed on the wrong (new) identity                  | Correlation didn't match                                          | Check the emails match exactly and only one identity has that email. Move it via <i>Identities</i> → <i>Identity Correlation</i> . |
| Changes in the app editor vanished                                   | Application not saved                                             | Click <b>Save</b> on the main application page                                                                                     |

---

## Appendix A: Building the operations by hand

---

Use this if you'd rather click through the UI than import XML. In **Applications** → **Google Analytics** → **Configuration**, click **Add Operation** for each one below. Fill in the tabs, click **Save** in the operation, and when all are done click **Save** on the application page.

**Create Account** - Context URL: `/v1alpha/accounts/410795087/accessBindings` , Method: **POST** - Header tab: Key `Content-Type` , Value `application/json` - Body tab: Raw, leave empty - Response tab: Root Path `$` , Successful Response Code `200` , and **Response Attribute Mapping** (the first *Add Row*):

| Schema Attribute         | Attribute Path     |
|--------------------------|--------------------|
| <code>bindingName</code> | <code>name</code>  |
| <code>user</code>        | <code>user</code>  |
| <code>roles</code>       | <code>roles</code> |

- Before Rule: **Google Analytics - Build Access Binding**

**Add Entitlement** - Context URL: `/v1alpha/$plan.nativeIdentity$` (type the `$` signs exactly), Method: **PATCH** - Header: `Content-Type` = `application/json` . Body: Raw, empty. Response: code `200` , no mappings. - Before Rule: **Google Analytics - Build Access Binding**

**Remove Entitlement**: exactly the same as Add Entitlement.

**Delete Account** - Context URL: `/v1alpha/$plan.nativeIdentity$` , Method: **DELETE** - No header, no body. Response code `200` . No Before Rule.

**Provisioning policy**: under *Provisioning Policies* → *Create Account* → *Add Field*: - Name `user` , Display Name `Google Email` , Type **String**, Required ☒ , Dynamic ☐ - Value: **Script**, `return identity.getEmail();`

**Correlation**: under *Correlation* → *New*: Application Attribute `user` → Identity Attribute `email` .

---

## Appendix B: Checking the OAuth scope

The scope isn't shown on the application form, so you check it on the Debug page.

1. **Close the application editor first**. Otherwise saving it later overwrites your change.
2. Go to `http://localhost:8080/identityiq/debug` , choose **Application**, and open **Google Analytics**.
3. Press Ctrl+F and search for `scope` . It must be: `https://www.googleapis.com/auth/analytics.manage.users` (**not** ending in `.readonly` , which can read but not grant access).
4. If you changed it, click **Save**, reload, and check again.